

Machine Learning in R: Overview and Keras Introduction

EDSD workshop on Machine Learning

Andrea Nigri, PhD

June 11, 2025

University of Foggia,
andrea.nigri@unifg.it

Overview

- R provides several powerful packages for Machine Learning.
- These packages cover a wide range of tasks: classification, regression, clustering, neural networks, etc.
- This presentation provides:
 - A quick tour of essential ML packages in R.
 - A detailed look at the Keras interface in R.

Popular ML Packages in R

- **caret** – Unified interface to many algorithms, including preprocessing and model tuning.
- **randomForest** – Implementation of the random forest algorithm.
- **xgboost** – High-performance gradient boosting.
- **keras** – Deep learning using TensorFlow backend in R.

Why Keras in R?

- Keras provides a high-level, user-friendly interface for building deep learning models.
- R users can leverage TensorFlow capabilities without switching to Python.
- Ideal for prototyping and educational use due to its simplicity.

Installing Keras in R

```
install.packages("keras")  
library(keras)  
install_keras() \\  
# Installs TensorFlow backend as well
```

Model Workflow in Keras

The typical Keras workflow involves:

1. **Define a model:** Sequential or Functional API.
2. **Add layers:** Dense, Dropout, Conv2D, etc.
3. **Compile:** Specify loss, optimizer, and metrics.
4. **Fit:** Train the model on data.
5. **Evaluate:** Assess model performance on test data.
6. **Predict:** Use the model to make predictions.

Key Keras Concepts

Understanding core components is essential to effectively use Keras:

- **Batch** – A subset of the training data passed through the network in one forward/backward pass.
- **Epoch** – One full pass over the entire training dataset.
- **Loss Function** – Measures how well the model performs; guides optimization.
- **Optimizer** – Algorithm that updates weights to minimize the loss (e.g., SGD, RMSprop, Adam).
- **Metrics** – Used to evaluate model performance (e.g., accuracy, mean absolute error).

Loss vs. Metrics

- **Loss Function:**

- Guides training via backpropagation.
- Examples: "mse", "categorical_crossentropy".

- **Metrics:**

- Human-understandable evaluation criteria.
- Examples: "accuracy", "mae", "precision".
- Do not affect model training.

What is a Batch?

- During training, data is split into small sets called **batches**.
- The model updates weights after each batch.
- **Batch Size:** Number of samples processed before updating model weights.
- **Trade-offs:**
 - Small batches: more updates, but slower training.
 - Large batches: faster but need more memory and might converge less well.

What is an Epoch?

- An **epoch** is one complete iteration over the full training dataset.
- Models are trained for multiple epochs.
- Each epoch consists of multiple batch updates.
- Monitor loss/metrics across epochs to evaluate learning progress.

Minimal Example: Dense Neural Network

```
library(keras)

model <- keras_model_sequential() %>%
  layer_dense(units = 64,
    activation = 'relu', input_shape = c(10)) %>%
  layer_dense(units = 1)

model %>% compile(
  loss = 'mse',
  optimizer = optimizer_rmsprop(),
  metrics = c('mae')
)
```

Minimal Example: Dense Neural Network

```
# Data
x_train <- matrix(runif(1000), nrow = 100)
y_train <- runif(100)

model %>% fit(x_train, y_train,
epochs = 10, batch_size = 10)
```

Summary

- R supports a wide range of ML tools, from classic algorithms to deep learning.
- keras bridges R and TensorFlow, enabling deep learning within R.
- Simple API, but powerful backend.
- Ideal for both experimentation and real-world applications.